Código:FT-DG-002
Versión:001
Fecha:08/06/2021

FORMATO DE DESARROLLO E INTEGRACIONES

CAPTURA FOTO

STF GROUP S.A.
INFORMATICA – Py. CREDI10 SOFTWARE PROPIO

Cali, lunes, 29 de enero de 2024

TABLA DE CONTENIDO

INTRODUCCION 3
1. DETALLE TECNICO 4
1.1 Detallar componentes existentes..... 4
1.2 Detallar funcionalidades 6
1.3 Observaciones y/o Comentarios.....**¡Error! Marcador no definido.**
2. CASOS DE PRUEBAS UNITARIAS
¡Error! Marcador no definido.

INTRODUCCION

Este documento muestra en detalle todas las actividades de la solución a desarrollar con sus dependencias, entradas y salidas. Así como los casos de pruebas unitarias realizadas a la solución.

NOTA: El desarrollo de cualquier solución de requerimiento debe seguir los estándares de desarrollo que define Arquitectura.

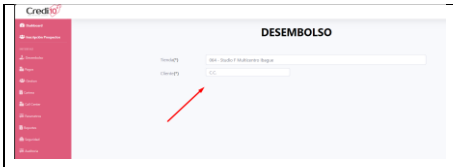
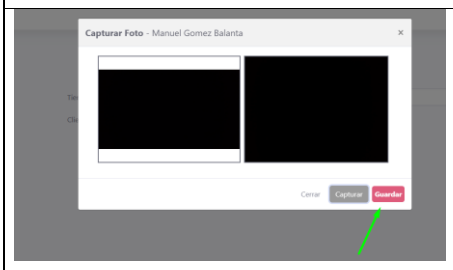
1. DETALLE TECNICO

1.1 Descripción del Requerimiento:

1.1.1 Requerimiento

Validar que el cliente tenga foto, si no, realizar la captura de la foto del cliente en el proceso antes de ingresar al formulario, también permitir recaptura en el formulario.

1.1.2 Flujo del Proceso

	Index.cshtml Se ingresa la información de la tienda y el cliente
	Camara.cshtml Se presiona en capturar para visualizar la foto que se captura en la parte derecha, si la foto sale mal puede volver a dar en capturar para actualizar la foto y la visualización
	Camara.cshtml Se presiona en guardar para confirmar la foto guardarla

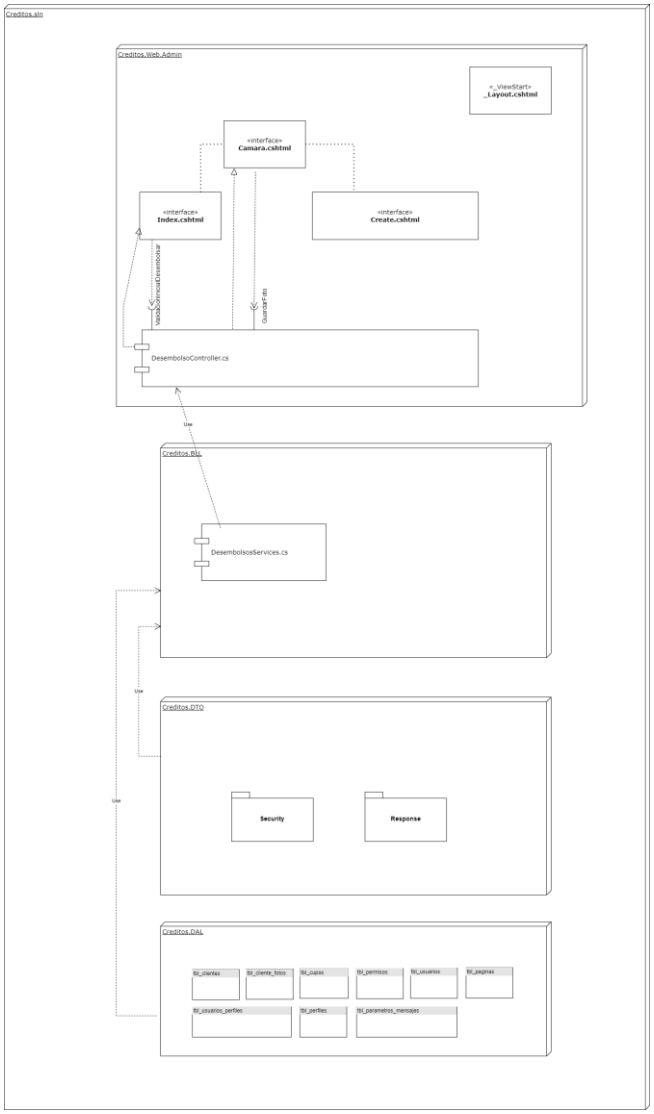
1.2 Detallar componentes existentes

SOLUCION

Creditos.Web.Admin • Controladores o DesembolsoController.cs • Vistas o Index.cshtml o Create.cshtml o Camara.cshtml	Clases y vista implementada para la captura de la foto.
Creditos.BLL • DesembolsosServices.cs	Clase implementada para realizar la validación de cliente tiene foto.
Creditos.DAL • ModelC10 o C10Model.edmx	Modelo de datos de entidad.
Creditos.DTO • OperationResponse	Clases de transferencia de datos
StyleLoader.css ,	Estilos pantalla carga
AutoNumeric.js	Librería Formato de números
Notification.js	Contiene los métodos que se utilizan para mostrar las alertas a los usuarios.
Bootstrap	Librería para el front v5.1.3
BASE DE DATOS	
tbl_clientes	Tabla
tbl_cliente_fotos	Tabla

1.3 Detallar funcionalidades

1.3.1 Diagrama de Componentes



Comentado [HA1]: Agregrar el diagrama de componentes
Usar: https://app.diagrams.net/

https://www.youtube.com/watch?app=desktop&v=JF7bH5Xo6ys

https://www.ionos.es/digitalguide/paginas-web/desarrollo-web/diagrama-de-componentes/

1.3.2 MER



1.3.3 Parametrizaciones

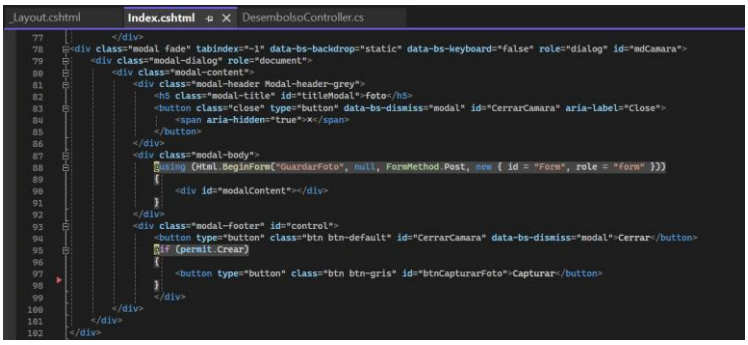
[tbl_parametros_mensajes], Campo Key_ valores: 'ClienteSinFoto', 'FotoGuardada', 'FotoNoGuardada'-

id_parametro	descripcion	grupo	key_	valor	codigo	estado
19	Cliente sin foto en validacion desembolso	Desembolso	ClienteSinFoto	El cliente no tiene foto	2004	1
57	Mensaje confirma foto guardada	Desembolso	FotoGuardada	Foto guardada con éxito	2042	1
58	Mensaje error guardando foto	Desembolso	FotoNoGuardada	La foto no se guardo	2043	1

Los anteriores registros son los mensajes que se muestran en el proceso de captura de la foto.

1.3.4 Proceso

La generación del crédito inicia con *Index.cshtml* del controlador *DesembolsoController.cs,* en esta vista se lanzan una serie de validaciones con el consumo de un endpoint incluida la validación de la existencia de al menos una foto del cliente. *Index.cshtml* tiene los inputs correspondientes a la tienda, el cliente y adicionalmente contiene un modal *#mdCamara* que es el que permite visualizar la vista Camara.cshtml para realizar la captura de la foto.



En el proceso de captura de fotos, el input relevante es de tipo texto *#NumeroDocumento*, donde se ingresa el número de documento del cliente. A través de JavaScript, se detecta cuándo se pierde el foco del campo de entrada, lo que desencadena la llamada a los métodos *'buscarDocumento'* y *'habilitarbtnValidar'*. Estos métodos realizan la validación del cliente y gestionan la habilitación del botón 'Aceptar'. Este botón permite al usuario continuar en el proceso de desembolso cuando se complete la validación requerida.

```
$('#NumeroDocumento').change(function (e) {
    clienteExiste = buscarDocumento();
    habilitarbtnValidar()

});
```

'buscarDocumento' captura el valor ingresado y se hace una solicitud mediante Ajax al endpoint *ValidacionInicialDesembolsar()* de *DesembolsoController.cs*, se envía en la carga útil el objeto *DatosFiltroDesembolsoDTO* con los datos ingresados en el filtro del *index.cshtml*, el endpoint *ValidacionInicialDesembolsar* retorna un objeto *OperationResponse*.

```
function buscarDocumento() {
    var elemento = $('#NumeroDocumento');
    var form = $('#Form');
    mostrarProgresoSimple();
    if (elemento.value != "") {
        $.ajax({
            url: '@Url.Action("ValidacionInicialDesembolsar", "Desembolso")',
            type : 'POST',
            data : form.serialize(),
            success: function (respuesta) {
                ocultarProgresoSimple();
                var TieneFoto = respuesta.Status;
                if (respuesta.MessageCode == ClienteSinFoto) {
                    var data = JSON.parse(respuesta.Data)
                    $('#NombreCliente').html(data.NombreCliente);
                    $('#btnAbrirCamara').click();
                    //FotoCapturada  Y clienteExiste  en este caso se establecen en el script de la vista de camara
                    if (!$('#btnAbrirCamara').length) {
                        // El elemento no existe
                        MostrarMensaje('El cliente no tiene foto asociada, su usuario no tiene permiso de captura de foto.', false);
                    }
                }

                else if (respuesta.Status) {
                    var data = JSON.parse(respuesta.Data)

                    $('#NombreCliente').html(data.NombreCliente);
                    FotoCapturada = true;
                    clienteExiste = true;
                    $('#btnAbrirCamara').hide();

                }
```

```
99+ referencias
public class OperationResponse
{
    99+ referencias
    public OperationResponse()
    {
        this.Cedulas = new List<ced>();
        this.Ids = new List<long>();
    }
    99+ referencias
    public string Message { get; set; }
    99+ referencias
    public int MessageCode { get; set; }
    99+ referencias
    public bool Status { get; set; }
    99+ referencias
    public bool IsError { get; set; }
    99+ referencias
    public bool Completed { get; set; }
    29 referencias
    public int Code { get; set; }
    99+ referencias
    public string Data { get; set; }
    84 referencias
    public byte[] Content { get; set; }
    1 referencia
    public List<long> Ids { get; set; }
    16 referencias
    public List<ced> Cedulas { get; set; }

}

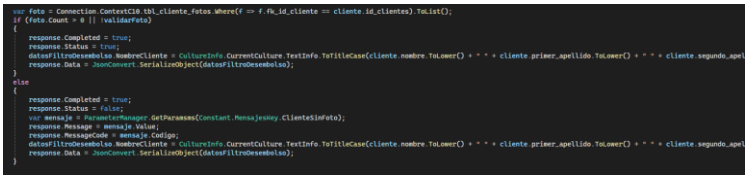

16 referencias
public class ced
{
    20 referencias
    public string cedula { get; set; }
}
```

El endpoint *ValidacionInicialDesembolsar* a través de DesembolsosServices y su metodo BLL ValidacionInicialDesembolsar verifica si el cliente es apto para realizar desembolso en el siguiente orden:

1. El cliente tenga datos de contacto.
2. El cliente tenga el cupo aceptado o que el cupo este sin aceptar pero que pertenezca a un preaprobado por gerencia.
3. El cliente tenga el cupo en estado vigente(10)
4. El cliente tenga cupo disponible. Se valida con el resultado del procedimiento almacenado *sp_cupo_disponible*
5. El cupo no supere el máximo de días de mora permitido (30) Se valida con el procedimiento almacenado *sp_cupo_en_mora*
6. El cliente tenga foto asociada en *tbl_cliente_fotos*.

Comentado [HTCA2]: Mencionar el numero de dias de mora permitido y si esta de pronto en algun parametro

Asumiendo que el cliente cumple con las condiciones de la 1 a la 5 se ejecuta la validación número 6 que consiste en que debe existir al menos una foto relacionada al cliente en [*tbl_cliente_fotos],* la variable *'validarFoto'* se utiliza para evitar la validación de la existencia de al menos una foto ya que el método también es usado por el controlador de los desembolsos e-commerce y *'validarFoto'* se utiliza para omitir la validación.



Si el cliente no cumple con la validación 6 se retorna el mensaje y el código del *key_ 'ClienteSinFoto' en el response de ValidacionInicialDesembolsar,* en la vista se verifica el *MessageCode* y si el valor corresponde al de la constante 'ClienteSinFoto = 2004' (de Constants.js) entonces simula un clic con JQuery sobre el botón *#btnAbrirCamara,* conn esta acción se captura el número de documento, el nombre del cliente y se carga en el modal *#mdCamara* la vista Camara.cshtml

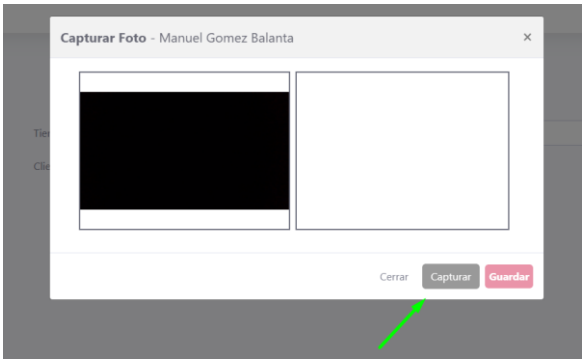


Nota:

- si el modal se cierra se habilita el *#btnAbrirCamara* o 'Tomar Foto' en el index para que pueda volver a abrir el modal de la cámara y continuar con el proceso de captura.
- *#btnAbrirCamara* o 'Tomar Foto' también está presente en la vista create.cshtml que tiene un modal idéntico a *#mdCamara de index.cshtml para* permitir volver a realizar la captura en la pantalla del formulario de desembolso.

```
$("#btnAbrirCamara").click(function (e) {
        e.preventDefault();
        var numeroDocumento = $('#NumeroDocumento').val();
        numeroDocumento = numeroDocumento.replaceAll(",", "");
        if (numeroDocumento.length > 0) {
            var NombreCliente = $('#NombreCliente').text();
            $("#Form").attr("action", "@Url.Action("Camara", new { key = ViewBag.Key })");
            $("#modalContent").html("");
            $("#titleModal").html('<strong>Capturar Foto </strong> <span>  -  </span>' + NombreCliente + '');
            $("#modalContent").load($(this).attr("href"));
            $("#mdCamara").modal("show", { backdrop: 'static', keyboard: false });
        } else {
            MostrarMensaje(ingreseCedulaCliente)

        }
    });
```



A continuación, se describe la vista Camara.cshtml, que se abre en el modal y contiene los elementos y el JavaScript necesarios para llevar a cabo la captura de la foto:

Elementos HTML:
<video> y <canvas>: Utilizados para mostrar el flujo de video en vivo desde la cámara y para representar la foto capturada, respectivamente. Se identifican por los IDs *#video* y *#canvas*.

Botones de control:
Dos botones permiten al usuario interactuar con la funcionalidad. El botón de captura se identifica como *#btnCapturarFoto* o Capturar, mientras que el botón para guardar la foto capturada se identifica como *#btnGuardarFoto o Guardar*.

Inicialización y configuración:

Inicialización de la cámara: Al cargar la página, se invoca la función *'init()'*, que utiliza la API '*navigator.mediaDevices.getUserMedia()'* para solicitar permiso al usuario para acceder a la cámara del dispositivo. El flujo de video se muestra en el elemento <video>.

Configuración de parámetros:
Se establecen las restricciones de la cámara, como el tamaño de la imagen, mediante el objeto constraints.

Captura y visualización de la foto:

Captura de foto: Cuando el usuario hace clic en el botón de captura *#btnCapturarFoto*, se llama a la función '*btnCapturarFoto.addEventListener()'* que utiliza el método *'drawImage()'* del contexto del canvas para capturar una instantánea del video en vivo y mostrarla en el canvas.

Visualización de la foto: La foto capturada se muestra en el canvas en tiempo real, permitiendo al usuario revisar y confirmar su calidad antes de guardarla.

Interacción del usuario:

Habilitación de Botones: El botón de guardar *#btnGuardarFoto* se deshabilita inicialmente y se activa solo después de que se captura una foto, lo que se logra mediante el uso de *$('#btnGuardarFoto').prop('disabled', false).*

Envío y procesamiento en el servidor:

Después de que el usuario ha capturado una foto y desea guardarla, esta se envía al servidor para su procesamiento y almacenamiento. El proceso de envío se realiza utilizando la API FormData() de JavaScript para crear un objeto FormData que contiene la imagen en formato blob, que es un objeto binario que representa los datos de la imagen.

Creación del objeto FormData:

Al hacer clic en el botón de guardar *#btnGuardarFoto*, se crea un objeto FormData utilizando var formData = new FormData().

Conversión de la imagen a blob:

La foto capturada se convierte en formato blob mediante el método canvas.toBlob(), que toma una función de devolución de llamada como argumento. Esta función de devolución de llamada recibe el blob generado como parámetro y lo agrega al objeto FormData creado anteriormente.

Agregar la imagen al FormData:

El blob generado se agrega al objeto FormData utilizando el método formData.append(). La imagen blob se adjunta junto con el número de cedula, lo que permite al servidor identificar y procesar correctamente la imagen recibida.

Envío de la solicitud al servidor:

Una vez que el objeto FormData está listo con la imagen adjunta, se realiza una solicitud AJAX utilizando $.ajax() para enviar el FormData al servidor. En la configuración de la solicitud AJAX, se especifica la URL del punto final en el servidor (@Url.Action("GuardarFoto")), el tipo de solicitud (POST), y se incluye el FormData como el cuerpo de la solicitud.



```
[HttpPost]
0 referencias
public ActionResult GuardarFoto()
{

    string UserName = SessionUser.UserName;
    var response = Desembolsos.GuardarImagenFoto(Request, UserName);
    Security.ControlAuditoria(SessionUser.UserName, SecurityUtil.GetIP(), Constant.ControlAuditoriaAcciones.GuardadoFoto, response.Message);
    return Json(response);

}
```

Procesamiento de la imagen en el servidor:

Extracción de información del Archivo:
Se extrae información del archivo recibido, como su nombre (cedula del cliente) y contenido

Generación de ruta de almacenamiento y almacenamiento en el sistema de archivos:
Se genera una ruta de almacenamiento en función de la información extraída del Request, se verifica si el directorio de fotos del cliente existe, si no existe se crea, y se almacena la foto en el directorio. La ruta del directorio del cliente es construida a partir de lo parametrizado en la *[tbl_parametros],* de esta tabla se toma la columna *value*, de los *key_ RutaArchivosClientes* y *CarpetaActualArchivosClientesSemestre* a los cuales se les concatena la cedula del cliente encriptada con sha256(), por lo que la ubicación de los archivos está dada por:

RutaArchivosClientes + CarpetaActualArchivosClientesSemestre + "\\" + Sha256(cedulaCliente)

Se determinó crear una carpeta por cada semestre del año para que no se tenga un directorio con infinitos subdirectorios, ya que a medida que va aumentando el número de subdirectorios la lectura es lenta.

Se guarda el archivo en el sistema de archivos utilizando la ruta generada previamente.

Registro en la Base de Datos:

Se registra la información de la imagen en la base de datos en la tabla [tbl_cliente_fotos].

```
public OperationResponse GuardarImagenFoto(HttpRequestBase requestBase, string usuario)
{
    var response = new OperationResponse();
    try
    {
        if (requestBase.Files.Count > 0)
        {
            var fechaHoraCaptura = DateTime.Now;
            var horaModificada =  Regex.Replace(fechaHoraCaptura.ToString("s"), "[\\:\\.\\- T]", "");
            horaModificada = Regex.Replace(horaModificada, @"\s+", String.Empty);
            var file = requestBase.Files[0];
            var fileId = requestBase.Files[1];
            var numeroIdentificacion = (Path.GetFileName(file.FileName));
            numeroIdentificacion = !string.IsNullOrEmpty(numeroIdentificacion) ? numeroIdentificacion.Replace(",", "").Replace(".", "") : "";
            var rutaBD = ParameterManager.GetParam(Constant.ParamKey.RutaArchivosClientes).Value;
            var NombreCarpetaVariable = ParameterManager.GetParamNow(Constant.ParamKey.CarpetaActualArchivosClientesSemestre).Value;
            var fullPath = rutaBD + NombreCarpetaVariable + '\\' + SecurityUtil.Sha256(numeroIdentificacion.Replace(".", "")) + "\\Fotos";

            if (!Directory.Exists(fullPath))
            {
                Directory.CreateDirectory(fullPath);
                //Directory.CreateDirectory(ruta);
                response.Status = true;
            }

            var nombreArchivo = horaModificada + ".jpg";
            Stream stream = file.InputStream;
            string fotoPath = fullPath + "\\" + nombreArchivo;//Guardar esto en la ruta de la base de datos
            file.SaveAs(fotoPath);

            var cliente = Connection.ContextC10.tbl_clientes.FirstOrDefault(c => c.numero_identificacion == numeroIdentificacion);
            var foto = new tbl_cliente_fotos()
            {
                fk_id_cliente = cliente.id_clientes,
                fecha_captura = fechaHoraCaptura,
                nombre_archivo = fotoPath.Replace(rutaBD, string.Empty),
                principal = true,
                usuario_creacion = usuario,
            };

            var fotos = Connection.ContextC10.tbl_cliente_fotos.Where(f => f.fk_id_cliente == cliente.id_clientes).ToList();

            foreach (var item in fotos)...

            if (fotos.Count == 3)...
            else
            {
                Connection.ContextC10.tbl_cliente_fotos.Add(foto);
                Connection.ContextC10.SaveChanges();
                response.Message = "Foto Guardada";
                response.Completed = true;
                response.IsError = false;
                response.Status = true;
                response.Data = fotoPath;
            }
        }
    }
```

Lo anterior presenta una visión completa de cómo se captura, procesa y almacena una foto del cliente desde la captura inicial del flujo de video en vivo hasta el procesamiento y almacenamiento de la imagen en el servidor.

1.3.5 Permisos de la funcionalidad

Comentado [HA3]: Agregar detalle de los permisos que se utlizan en esta funcionalidad

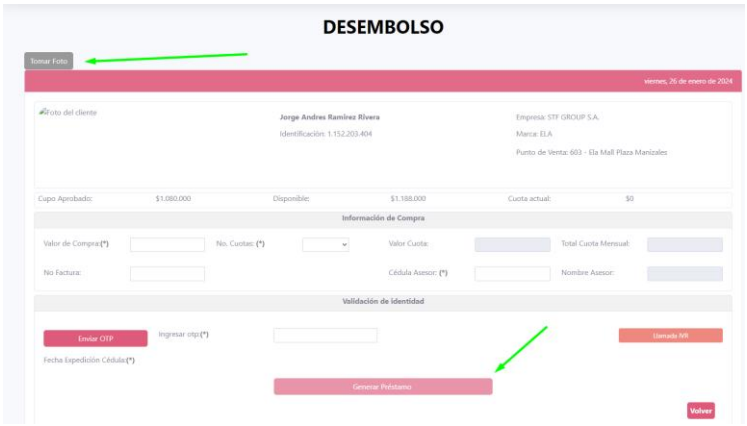
La funcionalidad de capturar la foto se administra con el mismo permiso de generar crédito solo los usuarios con la tarea de generar los créditos puedan realizar la operación de capturar la foto.

La operación de generar el crédito y tomar la foto del cliente se administra con la columna ‘crear’ de *[tbl_permisos]* si tiene asignado en true alguna otra columna de la tabla

[tbl_permisos] y la columna ‘crear’ está en false entonces el usuario podrá visualizar las vistas de desembolso(index.cshtml, create.csthml), pero no se le permitira tomar la foto



Si la columna ‘crear’ se encuentra en True entonces en la vista de generar el desembolso se muestran el botón ‘Tomar Foto’.



1.3.6 Log

Comentado [HA4]: Agregar el detalle de lo que se registra en el log

La columna ‘acción’ indica el proceso que se está realizando con el cliente y la columna ‘detalle’ indica el resultado de dicha acción o proceso, a continuación, se muestra la acción registrada en el log para el proceso de guardado de la foto:

accion	detalle
Guardar Foto	Resultado de guardado

La siguiente es la consultas del log del proceso de guardar la foto y deben ser buscadas con el login del usuario del sistema

```
SELECT  * FROM tbl_logs WHERE accion = 'Guardar Foto' and login = '<login
usuario que realiza el proceso>'
```

1.3.7 Lista de Scripts y la descripción de lo que contiene cada uno